

5.4 Itens: importação em massa via planilha XLSX — status de entrega

Bloco 5.4 Itens: importação em massa via planilha XLSX · Sprint 5 — Itens (o coração do sistema) · Entregue em 2026-09-22, ajustado em 2026-09-23 · Branch: `feature-dev-breno-automacao` · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`)

Resumo

Bloco concluído. Template XLSX para download, upload com parse no navegador, preview com linhas válidas em verde e com erro em vermelho, seleção do que importar, e inserção em lote transacional com numeração automática.

1. As seis atividades do bloco

ATIVIDADE DO CLICKUP	ESTADO
Template XLSX para download, com as colunas corretas e uma linha de exemplo	<code>/api/template/itens</code>
Upload da planilha preenchida e parse com exceljs	no navegador
Preview: válidas em verde, com erro em vermelho e o motivo	<code>itens-importar.tsx</code>
Permitir corrigir ou ignorar as linhas com erro antes de confirmar	checkbox por linha
Inserção em lote com numeração automática e transação	<code>importarItens</code>
Relatório final: X itens importados, Y ignorados	toast do resultado

Arquivos: `src/app/api/template/itens/route.ts` , `itens-importar.tsx` , e a action `importarItens` em `itens-actions.ts` . O schema de validação é o do 5.3 — `src/lib/itens-form.ts` .

2. Decisões

2.1 O template e o parse leem a MESMA lista de colunas

`COLUNAS_IMPORTACAO` , em `src/lib/itens-form.ts` , define título, largura e exemplo de cada coluna. O template é gerado dela, o parse procura por ela, e a validação usa o mesmo schema do formulário.

A consequência que importa: **não existe a possibilidade de o arquivo que a pessoa baixa ter cabeçalho diferente do que o sistema espera**. Acrescentar um campo é uma linha nessa lista, e template, parse e validação acompanham.

E o template **não traz** Área m² nem Valor total. A asserção de runtime confere as colunas que devem existir; as geradas ficam de fora porque o banco as calcula.

2.2 Parse no navegador, não no servidor

O arquivo não precisa subir para a pessoa descobrir que a coluna Quantidade está vazia. Planilha errada não gasta requisição nem chega perto do banco. O que sobe é só a lista de linhas já validadas.

2.3 Cabeçalho procurado, não assumido

O template abre com cinco linhas de instrução, e quem edita no Excel costuma acrescentar mais. O parse **procura** a linha que tem uma célula `Nº` — em qualquer coluna — em vez de assumir posição fixa, e monta um mapa título → coluna pela ordem real do arquivo — então **uma coluna movida no Excel não desalinha a importação em silêncio**. (Na entrega original só a coluna A era olhada; ver o fechamento.) Coluna ausente é erro explícito, com o nome da que falta.

Linha totalmente vazia é tratada como separador, não como erro: planilha editada à mão tem buraco no meio.

2.4 Número repetido dentro do próprio arquivo é acusado no preview

`validarPlanilha` acumula os números conforme as linhas passam. Sem isso, duas linhas com o mesmo número passariam no preview e a segunda estouraria o unique parcial no meio do insert. Números já existentes no banco entram no acumulador desde o início.

2.5 Transação: o lote inteiro volta atrás

O PostgREST executa `insert` com array de linhas como **uma** instrução, então ou entram todas ou nenhuma. É a transação que o enunciado pede, sem precisar de função SQL.

Se uma linha violar o unique, o lote inteiro volta — e é o comportamento desejado: **importação parcialmente aplicada é pior que importação recusada**, porque ninguém sabe onde parou. Há passo de escrita provando que nenhuma linha entra quando uma colide.

2.6 Numeração automática continua a sequência, inclusive dentro do lote

Linha sem número recebe `maior + 1`, calculado com `proximoNumeroItem` — o mesmo helper da tabela do 5.2 — a partir do que já existe na proposta e do que o próprio lote já consumiu.

2.7 Limite de 500 linhas

O corpo da Server Action é serializado, e planilha de 10 mil linhas travaria o navegador antes de chegar ao banco. Acima de 500 a action recusa com a mensagem pedindo para dividir o arquivo. Medido no fechamento da sprint: 500 linhas entram em 1.087 ms.

3. Números reais do `npm run validar`

Rodado em 2026-09-22 contra gc-dev.

CAMADA	ANTES DO 5.3/5.4	AGORA
estático	limpo	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint warnings or errors</code> ok

CAMADA	ANTES DO 5.3/5.4	AGORA	
unitário	85 casos	106 casos, 0 falhas	+21 (5.3/5.4)
build	31 rotas	32 rotas	+1
runtime	43/43 rotas	44/44 rotas ok	+1
dados	15/15 checagens	15/15 checagens ok	=
escrita	61/61 passos	73/73 passos ok	+12
navegador	34/34 passos	41/41 passos ok, console limpo	+7 (5.3/5.4)

Rota nova: `/api/template/itens`. `bash scripts/validar.sh build --aceitar-rotas` regravou `scripts/rotas-esperadas.txt` de **31 para 32**.

3.1 A asserção de runtime da rota

A entrada em `validacao-rotas.json` usa `esperaContentType: spreadsheetml` e `esperaXlsx`, e **abre a planilha de verdade** para conferir que as colunas `Nº`, `Tipo`, `Descrição`, `Quantidade`, `Unidade (QTD ou M2)` e `Valor unitário` estão no cabeçalho. Não é só content-type: é o arquivo.

3.2 Os passos de escrita da importação

`importarItens` grava o lote · o relatório traz importados e ignorados (2 e 3) · linha sem número recebe numeração automática · colunas GENERATED calculadas no lote (R\$ 200,00 e 2 m²) · **lote com número repetido é recusado inteiro** · **nenhuma linha do lote recusado entrou** (a prova da transação) · lote com unidade inválida é recusado pela action antes do banco · **visualizador não importa planilha**.

3.3 Os casos unitários da importação

Linha válida convertida; célula de unidade vazia caindo em `QTD` em vez de reprovar a linha; linha inválida devolvendo erro com o **título** da coluna e não a chave (`Quantidade:`, não `quantidade:`); numeração pela linha real da planilha (ver o fechamento); número repetido dentro do arquivo; número já existente no banco; várias linhas sem número convivendo; e o resumo do preview.

Fechamento da sprint 5 — três bugs e a cobertura que faltava

Revisão de 2026-09-23: os seis documentos da sprint foram relidos, e cada pendência nominal foi corrigida ou justificada. Números do plano inteiro no fim da revisão:

CAMADA	BASELINE (SPRINT 4)	FIM DA SPRINT 5
estático	limpo	<code>tsc --noEmit</code> limpo, ✓ No ESLint warnings or errors
unitário	51 casos	140 casos, 0 falhas

CAMADA	BASELINE (SPRINT 4)	FIM DA SPRINT 5
build	31 rotas	32 rotas, iguais à baseline
runtime	41/41 rotas	47/47 rotas ok
dados	12/12 checagens	18/18 checagens ok
escrita	38/38 passos	101/101 passos ok
navegador	24/24 passos	50/50 passos ok, console limpo

Rodado em 2026-09-23 contra gc-dev. `migration list` : 17/17 alinhadas. Depois da rodada, conferido por query: 0 propostas de teste, 0 contratos de teste, 0 itens órfãos, 0 arquivos de item no bucket.

A maior lacuna do bloco — o parse no navegador sem cobertura — foi fechada, e fechá-la revelou três bugs:

1. **A linha de exemplo do template era importada como item.** O comentário no código dizia que o preview a recusava; não recusava — ela é uma linha válida e saía em verde. Agora a Observação do exemplo começa com `[EXEMPLO]` e o preview a recusa com "É a linha de exemplo do template: apague ou sobrescreva antes de importar".
2. **Mover a coluna N° no Excel fazia a importação inteira ser recusada.** O cabeçalho só era procurado na coluna A. Agora é procurado em qualquer célula.
3. **O preview mostrava a linha errada.** Numerava a partir de 2, como se o cabeçalho estivesse na linha 1; no template ele está na 7. "Linha 2" do preview era a linha 8 do Excel. Agora o parser devolve a linha real.

Como ficou a cobertura:

- O parse saiu do componente para `src/lib/itens-planilha.ts`, função pura sobre um `ArrayBuffer`. 13 casos com `.xlsx` gerados de verdade: o layout do template, a linha de exemplo, coluna movida, fórmula, rich text, hiperlink, linha vazia no meio, cabeçalho ausente, coluna faltando, arquivo que não é planilha, 500 linhas, e a numeração real.
- **A camada navegador sobe uma planilha de verdade** pela tela: 4 linhas, 2 válidas, 2 com erro (o exemplo e uma quantidade zero), grava só as válidas, e o relatório diz "2 itens importados, 2 ignorados".
- **A runtime baixa o template real** da rota e o passa pelo parser real: 1 linha, recusada como exemplo. O teste unitário prova o parser; este prova que a rota entrega o layout que o parser entende.
- **O limite de 500 foi medido** (pendência 3): 500 entram em 1.087 ms; 501 são recusadas.
- **Fórmula e rich text** (pendência 4) têm teste.

As instruções do topo do template agora moram em `INSTRUcoes_TEMPLATE`, em `src/lib/itens-form.ts` — a rota e os testes leem do mesmo lugar.

4. O que NÃO foi validado — pendências nominais

1. **Ninguém abriu o template no Excel.** O conteúdo é provado (a runtime o lê com o parser real), mas a aparência — largura das colunas, as instruções do topo legíveis — só se confere abrindo o arquivo. É conferência manual.
2. **Não há como desfazer uma importação.** Importar 200 itens errados exige apagar um a um. É o bloco 5.7 (ações em lote).
3. **Corrigir uma linha com erro é "corrigir no Excel e subir de novo",** e não editar no preview. O enunciado admite as duas leituras ("corrigir ou ignorar"); editar no preview duplicaria o formulário do 5.3 dentro do modal. Decisão, não lacuna.

5. Próximo passo

5.5 Itens: foto do item. Depois, o 5.7 (ações em lote) é o que resolve a pendência 5 deste bloco.